

1. ¿Qué descubrieron Wiesel y Hubel sobre las neuronas que denominaron complejas o *complex cells* en inglés?

- ☒ Reciben información de células simples o simple cells para responder ante estructuras más complejas
- ☐ Al ser tan complejas, no se puede saber en absoluto cuál es su rol en la corteza visual.
- ☐ Procesan información básica como bordes rectos.
- ☐ Son células sin núcleo.

2. ¿Qué era el Neocognitron?

- ☐ Es una región de la corteza visual humana.
- ☒ Modelo computacional propuesto por Kunihiko Fukushima inspirado en el descubrimiento de Hubel and Wiesel.
- ☐ Modelo propuesto por Yan n LeCun y Yoshua Bengio.
- ☐ Tipo de células en la corteza visual encargadas de procesar bordes rectos.

3. ¿Cuál es la diferencia principal entre las técnicas tradicionales de machine learning y las técnicas de Deep Learning?

- ☐ No hay diferencia alguna.
- ☐ En las técnicas tradicionales de machine learning, el desarrollador tiene que dedicar una cantidad considerable de tiempo para realizar la ingeniería de características, mientras que en las técnicas de Deep Learning este tiempo en ingeniería de características es muy poco o ninguno.
- ☐ Las técnicas tradicionales de machine learning se implementan en R o Matlab, mientras que las de Deep Learning solamente se implementan en Python.
- ☐ En las técnicas de Deep Learning, el desarrollador tiene que dedicar una cantidad considerable de tiempo para realizar la ingeniería de características, mientras que en las técnicas tradicionales de Deep Learning este tiempo en ingeniería de características es muy poco o ninguno.

4. ¿Qué hace la función `torch.view()`?

- ☒ Cambia la forma del tensor sin alterar los datos.
- ☐ Computa el gradiente del tensor.
- ☐ Calcula la vista del tensor.
- ☐ Visualiza el tensor.

5. Observa el siguiente código fuente de una red neuronal para clasificación binaria:

```
import torch.nn as nn
import torch.nn.functional as F

class BinaryClassifier(nn.Module):
    def __init__(self):
        super(BinaryClassifier, self).__init__()
        self.linear1 = nn.Linear(2, 10)
        self.linear2 = nn.Linear(10, 1)

    def forward(self, x):
        x = F.relu(self.linear1(x))
        x = torch.sigmoid(self.linear2(x))
        return x
```

¿Cuántas capas tiene esta red neuronal, considerando solo las capas con parámetros entrenables?

- ☐ 1
- ☒ 2
- ☐ 10
- ☐ Ninguna de las otras opciones es correcta

6. Analiza el siguiente código:

```
import tensorflow as tf

# Creación de un tensor con valores aleatorios uniformes
tensor = tf.random.uniform(shape=(2, 3), minval=0, maxval=10)
print("Tensor creado:\n", tensor)
```

¿Qué afirmación es correcta sobre el tensor `tensor` generado por este código?

- ☒ Es un tensor de 2x3 con valores aleatorios entre 0 y 10.
- ☐ Es un tensor de 2x3 con valores normalmente distribuidos entre 0 y 10.
- ☐ Es un tensor de 2x3 con valores igualmente espaciados entre sí entre 0 y 10.
- ☐ Es un tensor de 3x2 con valores aleatorios entre 0 y 1.

7. Analice el siguiente código:

```
import tensorflow as tf
from tensorflow.keras import layers
import matplotlib.pyplot as plt

n_samples = 1000
n_features = 4
X = tf.random.normal(shape=(n_samples, n_features), mean=0, stddev=1)
weights = tf.constant([0.5, -0.3, 0.2, 0.1], dtype=tf.float32)
bias = tf.constant(0.1, dtype=tf.float32)
logits = tf.matmul(X, tf.reshape(weights, (-1, 1))) + bias
probs = tf.sigmoid(logits)
y = tf.cast(probs > 0.5, dtype=tf.float32)

train_size = int(0.8 * n_samples)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

modelo = tf.keras.Sequential([
    layers.Input(shape=(n_features,)),
    layers.Dense(units=8, activation='relu'),
    layers.Dense(units=1, activation='sigmoid')
])

modelo.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history = modelo.fit(
    X_train, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)

loss, accuracy = modelo.evaluate(X_test, y_test, verbose=0)
# 4. Evaluar el modelo
loss, accuracy = modelo.evaluate(X_test, y_test, verbose=0)
```

De este código se puede decir:

- ☐ Genera datos uniformes de 1000 muestras con 4 características y entrena un modelo de clasificación multiclase con 3 capas entrenables.
- ☒ Genera datos normalmente distribuidos de 1000 muestras con 4 características y entrena un modelo de clasificación binaria con 2 capas entrenables.
- ☐ Genera datos aleatorios de 800 muestras con 8 características y entrena un modelo de regresión con 1 capa entrenable usando una pérdida de error cuadrático medio.
- ☐ Genera datos sintéticos de 1000 muestras con 4 características y entrena un modelo de clasificación binaria con 8 capas entrenables optimizado con SGD.